

Distributed File System (DFS) — Marco 2

▯▯ Visão Geral

Este projeto implementa um **Sistema de Arquivos Distribuído (DFS)** em Python, utilizando **sockets TCP** para comunicação entre processos e **Protobuf** para serialização das mensagens.

No **Marco 2**, o sistema deixa de operar como um armazenamento centralizado e passa a trabalhar com uma arquitetura distribuída formada por:

- múltiplos nós de armazenamento;
- particionamento de arquivos por shard;
- coordenador responsável por rotear requisições;
- comunicação entre processos independentes;
- armazenamento local isolado por nó.

A meta deste marco é demonstrar que o sistema consegue distribuir os arquivos corretamente entre os nós, manter a execução organizada e preparar a base para os próximos marcos, como replicação, consistência e tolerância a falhas.

▯▯ Arquitetura Geral

O DFS foi organizado em camadas, de forma a separar bem responsabilidades e facilitar evolução futura.

Camadas principais

- **Cliente (CLI)**
Responsável por interpretar os comandos do usuário e enviar as requisições ao sistema.
- **Coordenador**
Recebe as requisições da CLI, calcula o shard do arquivo e encaminha a operação para o nó correto.
- **Nós de armazenamento**
Cada nó mantém seu próprio armazenamento local e executa as operações de forma independente.

Fluxo principal

CLI → Coordenador → Nó responsável → Storage local

Visão dos componentes

- o **CLI** conversa com o coordenador;

- o **coordenador** decide onde cada arquivo deve ficar;
 - o **shard manager** define a regra de distribuição;
 - o **node registry** mantém os nós cadastrados;
 - o **node client** faz a comunicação interna entre coordenador e nós;
 - o **node service** executa as operações dentro de cada nó;
 - o **local storage** grava, lê, remove e lista arquivos no disco;
 - o **protocol** traduz objetos Python em bytes Protobuf e vice-versa;
 - o **frame** garante que as mensagens TCP sejam lidas corretamente.
-

□□ Funcionalidades

O sistema suporta as operações básicas esperadas de um DFS:

- **PUT**: envia um arquivo local para o DFS;
- **GET**: recupera um arquivo do DFS para a máquina local;
- **RM**: remove um arquivo do DFS;
- **LIST**: lista os arquivos distribuídos entre os nós.

Comportamento das operações

- **PUT**: o coordenador calcula o shard do caminho lógico e encaminha o arquivo ao nó responsável;
 - **GET**: o coordenador localiza o nó dono do arquivo e solicita seu conteúdo;
 - **RM**: o arquivo é removido no nó responsável;
 - **LIST**: o coordenador consulta todos os nós e consolida as respostas em uma listagem única.
-

□□ Conceitos Implementados

Sharding

A distribuição dos arquivos é feita com base no hash do caminho lógico:

`hash(path) % número_de_nós`

Isso garante que:

- o mesmo caminho sempre vá para o mesmo nó;
- a distribuição seja determinística;
- o balanceamento inicial seja simples e previsível.

Node ID

Identifica o nó associado a uma operação ou resposta.

Esse campo ajuda na rastreabilidade das requisições dentro do cluster.

Shard ID

Representa a partição lógica usada para decidir onde o arquivo será armazenado.

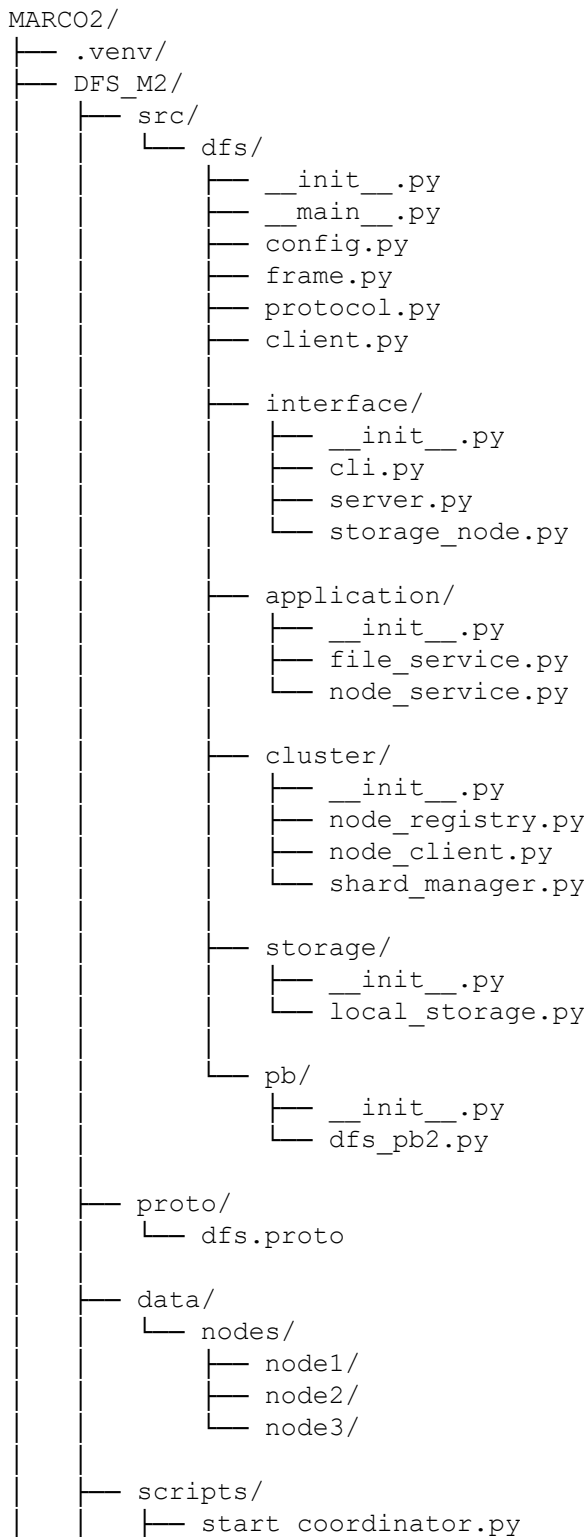
Comunicação TCP

O projeto usa **sockets TCP diretos**, sem gRPC em runtime.
O Protobuf é usado apenas como formato de serialização das mensagens.

Framing

Como o TCP trabalha como fluxo contínuo de bytes, o projeto utiliza framing por tamanho para garantir que cada mensagem seja lida corretamente e sem mistura com outras.

📁📁📁 Estrutura do Projeto



```
├── start_node1.py
│   ├── start_node2.py
│   └── start_node3.py
├── requirements.txt
└── README.md

run_cluster.py
run_cli.py
teste.txt
```

📁 O que faz cada arquivo

Raiz da pasta **MARCO2/**

- **run_cluster.py**
Script lançador que sobe automaticamente os três nós e o coordenador.
- **run_cli.py**
Script lançador da CLI. Ele permite usar os comandos do DFS sem precisar entrar manualmente na pasta `DFS_M2`.

Pasta **DFS_M2/src/dfs/**

- **__main__.py**
Ponto de entrada do pacote. Permite rodar a CLI com `python -m dfs`.
- **config.py**
Centraliza portas, hosts, caminhos e a configuração dos nós do cluster.
- **frame.py**
Implementa o framing das mensagens TCP por tamanho.
- **protocol.py**
Traduz as mensagens Protobuf entre bytes e objetos Python.
- **client.py**
Cliente TCP usado pela CLI para falar com o coordenador.

Pasta **interface/**

-

cli.py

Interface de linha de comando. Interpreta `put`, `get`, `rm` e `list`.

-

server.py

Coordenador principal do DFS. Recebe as requisições da CLI e roteia para o nó correto.

-

storage_node.py

Servidor de um nó de armazenamento individual. Cada instância escuta em uma porta própria.

Pasta application/

-

file_service.py

Camada de serviço do coordenador. Decide o destino da operação e faz o encaminhamento.

-

node_service.py

Camada de serviço que roda dentro de cada nó e executa as operações localmente.

Pasta cluster/

-

node_registry.py

Mantém a lista de nós disponíveis, seus hosts, portas e diretórios.

-

shard_manager.py

Calcula o shard responsável por cada caminho lógico.

-

node_client.py

Cliente interno que o coordenador usa para se comunicar com um nó.

Pasta storage/

- **local_storage.py**

Implementa o armazenamento local do nó: salvar, ler, apagar e listar arquivos.

Pasta pb/

- **dfs_pb2.py**

Arquivo gerado automaticamente pelo Protobuf a partir de `dfs.proto`.

Pasta proto/

- **dfs.proto**

Define a estrutura das mensagens `FileRequest` e `FileResponse`.

Pasta `scripts/`

- `start_coordinator.py`
Script auxiliar para subir o coordenador.
 - `start_node1.py`
Script auxiliar para subir o nó 1.
 - `start_node2.py`
Script auxiliar para subir o nó 2.
 - `start_node3.py`
Script auxiliar para subir o nó 3.
-

📄 Como Executar

A execução foi pensada para ser feita a partir da pasta `MARCO2/`.

1. Criar o ambiente virtual

Na pasta `MARCO2/`:

```
python -m venv .venv
```

2. Ativar o ambiente virtual

Linux / macOS

```
source .venv/bin/activate
```

Windows (PowerShell / CMD)

```
.venv\Scripts\activate
```

Windows com Git Bash

```
source .venv/Scripts/activate
```

3. Instalar as dependências

Com a venv ativada:

```
pip install -r DFS_M2/requirements.txt
```

4. Gerar os arquivos do Protobuf

Sempre que o arquivo `DFS_M2/proto/dfs.proto` for alterado, regenere o código Python:

```
cd DFS_M2
python -m grpc_tools.protoc -I=proto --python_out=src proto/dfs.proto
```

Depois volte para a pasta `MARCO2/` se necessário:

```
cd ..
```

5. Garantir que os diretórios dos nós existam

Se ainda não existirem:

```
mkdir DFS_M2/data/nodes/node1
mkdir DFS_M2/data/nodes/node2
mkdir DFS_M2/data/nodes/node3
```

6. Subir o cluster completo

Para evitar subir nó por nó manualmente, use:

```
python run_cluster.py
```

Esse script sobe:

- node1
- node2
- node3
- coordenador

Deixe esse terminal aberto enquanto estiver usando o DFS.

7. Usar o cliente (CLI)

Em outro terminal, com a venv ativada:

```
python run_cli.py <comando> [argumentos]
```

Exemplos:

```
python run_cli.py list
python run_cli.py put DFS_M2/teste.txt docs/teste.txt
python run_cli.py get docs/teste.txt copia.txt
python run_cli.py rm docs/teste.txt
```

❏ Exemplo de Uso

Criar um arquivo local

```
echo "teste distribuidos" > DFS_M2/teste.txt
```

Enviar para o DFS

```
python run_cli.py put DFS_M2/teste.txt docs/teste.txt
```

Listar arquivos

```
python run_cli.py list
```

Baixar um arquivo

```
python run_cli.py get docs/teste.txt copia.txt
```

Remover um arquivo

```
python run_cli.py rm docs/teste.txt
```

❏ Fluxos de Operação

PUT

CLI → Coordenador → Shard responsável → Nó → Disco local

GET

CLI → Coordenador → Nó responsável → Retorno do conteúdo

RM

LIST

Coordenador consulta todos os nós → consolida as respostas → retorna a listagem

☐☐☐ Decisões de Projeto

- uso de **socket TCP puro** para comunicação entre processos;
 - uso de **Protobuf** para serialização compacta e estruturada;
 - uso de **framing por tamanho** para resolver o problema do fluxo contínuo do TCP;
 - separação clara entre interface, rede, aplicação, cluster e armazenamento;
 - distribuição determinística dos arquivos por hash;
 - roteamento centralizado pelo coordenador;
 - execução independente dos nós de armazenamento.
-

☐☐ Critérios Atendidos no Marco 2

- múltiplos nós de armazenamento;
 - distribuição correta dos dados;
 - balanceamento inicial simples;
 - comunicação entre nós via socket;
 - estrutura pronta para expansão futura.
-

☐☐ Possíveis Problemas

- esquecer de regenerar o Protobuf após alterar o `dfs.proto`;
 - iniciar a CLI sem subir o coordenador e os nós;
 - portas já ocupadas por processos antigos;
 - diretórios dos nós inexistentes;
 - usar comandos da CLI sem o formato correto;
 - executar o `put` com caminho local inexistente.
-

☐☐ Próximos Passos

O projeto está preparado para evoluir para os próximos marcos:

- replicação de dados;
 - definição de consistência;
 - tolerância a falhas;
 - detecção por heartbeat;
 - re-replicação automática;
 - testes de escalabilidade.
-

🔗🔗🔗 Observações

- o sistema usa hashing determinístico para roteamento;
 - não há replicação no Marco 2;
 - cada arquivo pertence a um único nó responsável;
 - o armazenamento é local em cada nó;
 - a comunicação continua baseada em sockets TCP;
 - o coordenador consolida as respostas quando necessário;
 - o caminho local do arquivo deve existir antes do envio;
 - o caminho lógico informado no DFS pode ser diferente do caminho do arquivo na máquina local.
-

🔗🔗🔗 Autor

Higor Ferreira Silva
Matrícula: 202201635